UNIVERSITY OF TRANSPORT AND COMMUNICATIONS

HO CHI MINH CITY

INSTITUTE OF INFORMATION TECHNOLOGY AND ELECTRICAL - ELECTRONICS ENGINEERING

PROJECT REPORT

CPU SCHEDULING: FIRST COME, FIRST SERVED (FCFS) ALGORITHM

Course: Operating Systems

Course Code: 012012500106

Students in Charge

Full name	Student ID
1. Hoàng Việt Quang	066206005044
2. Nguyễn Văn Hiệp	066206009962
3. Nguyễn Văn Khương	066206009437
4. Nguyễn Tấn Lộc	064206011286
5. Đàm Triệu Vỹ	066206000409
6. Nguyễn Nhật Trường	064206005560
7. Hồ Văn Phát	064206001095

Ho Chi Minh City, October 2025

Table of contents

LIST OF ABBREVIATIONS ii
INTRODUCTION 1
CHAPTER 1: THEORETICAL BACKGROUND 2
1.1. First-Come, First-Served (FCFS) Scheduling 2
1.2. Key Metrics and Formulae 2
1.3. Short Comparisons and Practical Trade-offs 3
CHAPTER 2: ALGORITHM & IMPLEMENTATION 3
2.1. Algorithm Description 3
2.2. Pseudocode 4
2.2.1. Explanation: 4
2.3. Real code 4
2.4 Process Flow 8
2.5 Discussion 9
CHAPTER 3: EXPERIMENTAL RESULTS 9
3.1. Calculation Formulas for Time Metrics 9
3.2. Case 1 – Short Burst Time (Uniform Workload) 11
3.2.1. Description 11
3.2.2. Results and Observations 11
3.3. Case 2 – Long Burst Time (Unbalanced Workload) 12
3.3.1. Description 12
3.3.2. Results and Observations 13
3.4. Summary of Findings 14
CONCLUSION 14
REFERENCES 16

LIST OF ABBREVIATIONS

AT	Arrival Time
BT	Burst Time
CPU	Central Processing Unit
CT	Completion Time
FCFS	First-Come, First-Served
FIFO	First In, First Out
OS	Operating System
RT	Response Time
SJF	Shortest Job First
SRT	Shortest Remaining Time
TAT	Turnaround Time
WT	Waiting Time

INTRODUCTION

In modern multiprogramming operating systems, it is common for multiple processes to reside in memory simultaneously, ready for execution [1], [2]. However, on a system with a single-core or limited-core Central Processing Unit (CPU), only one process can run at any given time. This creates a fundamental need for a mechanism to efficiently manage and allocate CPU resources among these competing processes. This mechanism is known as CPU Scheduling [1].

The primary objective of CPU scheduling is to optimize system performance by balancing factors such as throughput, waiting time, turnaround time, and response time, while also ensuring fairness to all processes. To achieve this, a variety of different scheduling algorithms have been developed [1], [2].

Among the foundational scheduling algorithms, First-Come, First-Served (FCFS) is the most basic and intuitive [1], [2], [3]. As its name implies, FCFS operates exactly like a real-world queue: the process that requests the CPU first is allocated the CPU first. That process retains the CPU until it either terminates or voluntarily releases it (for example, to perform an I/O request). The core advantage of FCFS lies in its simplicity; it is very easy to understand and straightforward to implement using a basic FIFO (First-In, First-Out) queue [3]. Furthermore, FCFS is considered perfectly fair according to arrival order, as it guarantees that every process will eventually be executed and no process will suffer from starvation. However, this simplicity comes at a significant performance cost. The algorithm's greatest weakness is its poor average response time. This problem becomes particularly severe when a process with a long CPU burst (a “long job”) arrives before processes with short CPU bursts (“short jobs”). This phenomenon, known as the “convoy effect,” [1], [4] forces the short processes to wait idly for the long one to complete, leading to inefficient resource utilization and a degraded interactive user experience.

Recognizing the foundational role of FCFS as well as its inherent limitations, this report aims to achieve the following objectives:

1. Implementation: To build a simulator that correctly implements the FCFS scheduling algorithm.
2. Illustration: To use specific examples and scenarios to clearly demonstrate how FCFS operates, paying special attention to illustrating the convoy effect in the presence of long jobs.
3. Comparison: To conduct a quick comparison of FCFS against more advanced scheduling algorithms, such as Round Robin (RR) and Shortest Job First (SJF/SRTF), to highlight the trade-offs between simplicity and performance (particularly regarding waiting and response times).

CHAPTER 1: THEORETICAL BACKGROUND

1.1. First-Come, First-Served (FCFS) Scheduling

First-Come, First-Served (FCFS) is the most straightforward CPU scheduling policy: processes are placed in a FIFO (first-in, first-out) queue and are dispatched in the order they arrive [1], [2]. FCFS is non-preemptive — once a process starts it runs to completion — which makes the algorithm easy to reason about and simple to implement [3]. However, this simplicity comes at a cost. A phenomenon known as the convoy effect can occur when a long job blocks the CPU and forces many short jobs to wait behind it, substantially inflating average waiting times and harming overall responsiveness in mixed workloads. In practice, FCFS can be fair in a chronological sense but inefficient when job lengths vary widely.

1.2. Key Metrics and Formulae

To compare scheduling algorithms we rely on a small set of standard metrics that capture latency and service fairness [1], [2]. These are typically expressed as:

- Turnaround Time (TAT) = Completion Time (CT) – Arrival Time (AT)
- Waiting Time (WT) = Turnaround Time (TAT) – Burst Time (BT)
- Response Time = Start of First Run – Arrival Time (AT)

Turnaround time measures how long a job spends in the system from arrival to finish. Waiting time isolates the amount of time a job sits idle in the ready queue, and response time captures how quickly a job receives its first CPU allocation — a critical metric for interactive systems. These metrics together provide a concise picture of both throughput and user-perceived latency.

1.3. Short Comparisons and Practical Trade-offs

It is useful to position FCFS against a couple of other common policies to understand its practical trade-offs: [3], [4]

SJF / SRTF (Shortest Job First / Shortest Remaining Time First): By preferring shorter jobs, these policies can minimize average waiting time in idealized scenarios [1], [5]. However, they risk starvation for long jobs when short jobs arrive frequently — a theoretical optimum that may perform poorly for fairness in real systems.

Round Robin (RR): RR introduces a fixed time quantum so that each runnable process receives CPU time in turn [2], [3]. If the quantum is very large, RR tends toward FCFS behavior; if it is very small, context switching overhead rises and overall throughput may fall, though responsiveness improves for interactive tasks. Selecting the quantum is therefore a tuning exercise: balance responsiveness against context-switch cost.

In short, FCFS is attractive for its determinism and low overhead, but modern systems often favor preemptive schemes (or hybrids) to manage variability in job sizes and to meet responsiveness constraints. The Operating Systems: Three Easy Pieces (OSTEP) materials provide a compact framework to reason about these trade-offs and the metrics used above [1].

CHAPTER 2: ALGORITHM & IMPLEMENTATION

2.1. Algorithm Description

The **First Come First Serve (FCFS)** scheduling algorithm is one of the simplest CPU scheduling techniques, operating on the principle of **"first in, first out" (FIFO)** [1], [2], [3].

Processes are executed in the order of their **arrival time (AT)** — the process that

arrives first is allocated the CPU first. If the CPU is idle and no process has arrived yet, the system remains in an **IDLE** state until the next process enters the ready queue.

FCFS is a **non-preemptive** algorithm, meaning once a process starts execution, it runs until completion without interruption. This makes FCFS easy to implement and suitable for modeling queue-based systems.

2.2. Pseudocode

```
Start
  Sort all processes by Arrival_Time
  Time = 0
  For each process P in sorted order:
      If Time < P.Arrival_Time → Time = P.Arrival_Time
      P.Start = Time
      P.Completion = Time + P.Burst
      Time = P.Completion
End
```

2.2.1. Explanation:

Processes are sorted by arrival time.

If the CPU is idle (Time < Arrival Time), an IDLE period is inserted.

Start time (ST) and completion time (CT) are assigned for each process.

The following performance metrics are calculated:

Turnaround Time (TAT) = CT – AT

Waiting Time (WT) = TAT – BT

Response Time = ST – AT

2.3. Real code

```
class fcfs:

  def __init__(self, _processID = str, _arrivalTime = int, _burstTime = int):
```

```
        self.processID = _processID
        self.arrivalTime = _arrivalTime
        self.burstTime = _burstTime

        self.completeTime = []
        self.TAT = []
        self.WT = []
        self.avgTAT = 0.0
        self.avgWT = 0.0
        self.results = []

    def genProcessFlow(self):
        for i in range(len(self.arrivalTime)):

            min_idx = i
            for j in range(i+1, len(self.arrivalTime)):
                if self.arrivalTime[min_idx] > self.arrivalTime[j]:
                    min_idx = j

            self.arrivalTime[i], self.arrivalTime[min_idx] =
self.arrivalTime[min_idx], self.arrivalTime[i]
            self.processID[i], self.processID[min_idx] = self.processID[min_idx],
self.processID[i]
```

```
            self.burstTime[i], self.burstTime[min_idx] = self.burstTime[min_idx],
self.burstTime[i]


    def printProcessFlow(self):
        for i in range(len(self.processID)):
            print(self.processID[i], end = " --> ")
        print("end")


    def startScheduling(self):
        time = 0
        for i in range(len(self.processID)):
            time = max(time, self.arrivalTime[i])
            self.completeTime.append(time + self.burstTime[i])
            self.TAT.append(self.completeTime[i] - self.arrivalTime[i])
            self.WT.append(self.TAT[i] - self.burstTime[i])
            time = self.completeTime[i]



            self.results.append({
            "Process": self.processID[i],
            "AT": self.arrivalTime[i],
            "BT": self.burstTime[i],
            "CT": self.completeTime[i],
```

```
            "TAT": self.TAT[i],
            "WT": self.WT[i]
        })

    def calcuAvg(self):
        self.calcuAvgTAT()
        self.calcuAvgWT()

    def calcuAvgTAT(self):
        totalTAT = 0
        for tatVal in self.TAT:
            totalTAT += tatVal

        if len(self.processID) > 0:
            self.avgTAT = totalTAT / len(self.processID)
        else:
            self.avgTAT = 0

    def calcuAvgWT(self):
        totalWT = 0
        for wtVal in self.WT:
```

```
            totalWT += wtVal


        if len(self.processID) > 0:
            self.avgWT = totalWT / len(self.processID)
        else:
            self.avgWT = 0


    def printResults(self):
        print("--- FCFS scheduling results ---")
        print("{:<10} {:<5} {:<5} {:<5} {:<5} {:<5}".format("Process", "AT",
"BT", "CT", "TAT", "WT"))
        for p in self.results:
            print("{:<10} {:<5} {:<5} {:<5} {:<5} {:<5}".format(
                p["Process"], p["AT"], p["BT"], p["CT"], p["TAT"], p["WT"]
        ))


        print("\nAverage Turnaround Time (TAT): " + str(self.avgTAT))
        print("Average Waiting Time (WT): " + str(self.avgWT))
```

2.4 Process Flow

1. **Input Data:** Enter the number of processes and their Arrival Time (AT) and Burst Time (BT).
2. **Sort by Arrival Time:** Arrange processes in order of arrival.
3. **Time Simulation:** Run each process sequentially; insert *IDLE* if the CPU is free.

4. **Compute Results:** Calculate CT, TAT, WT, and Response Time for each process.
5. **Output Results:** Display process table and Gantt chart (details shown in *Experiment & Results* section).

2.5 Discussion

The FCFS algorithm is simple, intuitive, and easy to implement. However, it suffers from the **convoy effect**, where a long-running process delays all subsequent ones [1], [4]. Despite this drawback, FCFS remains an essential baseline for understanding and comparing more advanced scheduling algorithms such as **Shortest Job First (SJF)** and **Round Robin (RR)** [4], [5].

CHAPTER 3: EXPERIMENTAL RESULTS

After completing the implementation of the first-come, first-served CPU scheduling algorithm in Python, our team proposed two datasets to test the performance of this algorithm. These two datasets come with two favorable and unfavorable cases for the algorithm. To evaluate this performance, we need to pay special attention to Completion Time (CT), Processing Time (TAT) and Waiting Time (WT).

3.1. Calculation Formulas for Time Metrics

To evaluate the performance of the FCFS scheduling algorithm, several key timing parameters were calculated [1], [2]. The formulas used in the simulation are presented as follows:

- **Completion Time (CT):** Completion Time is a process finishes its execution.

(3.1.1)

Where:

AT: Arrival Time.
BT: Burst Time.
i: Process

- **Turnaround Time (TAT):** Turnaround Time is the total time taken for a process to complete after its arrival.

(3.1.2)

Where:

CT: Completion Time.
AT: Arrival Time.
i: Process.

- **Waiting Time (WT):** Waiting Time is the total time a process spends waiting in the ready queue before execution.

(3.1.3)

Where:

TAT: Turnaround Time.
BT: Burst Time.
i: Process.

- **Response Time (RT):** The time interval between the process arrival and the first CPU allocation.

(3.1.4)

Where:

ST: Start Time.
AT: Arrival Time.
i: Process.

Note: In FCFS, since processes are executed in the order of arrival and non-preemptive . **RT = WT**.

- **Average Waiting Time:**

(3.1.5)

- **Average Turnaround Time (Tavg):**

(3.1.6)

- **Average Response Time (Ravg):**

(3.1.7)

3.2. Case 1 – Short Burst Time (Uniform Workload)

3.2.1. Description

In this dataset, the processes all have short burst times and are almost similar. The processes arrive at almost the same time, thereby creating a uniform workload environment. Through this dataset, we can see that the CPU operates almost continuously without idle time. From that, we can conclude that in a uniform workload environment, the FCFS algorithm fully utilizes the CPU's performance.

3.2.2. Results and Observations

Table 3.2.2 – FCFS Scheduling Results (Short Burst Time – Uniform Workload)

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT)	Waiting Time (WT)
P00	0	1	1	1	0
P01	2	3	5	3	0
P04	2	2	7	5	3
P02	3	2	9	6	4
P05	3	1	10	7	6
P03	9	4	14	5	1

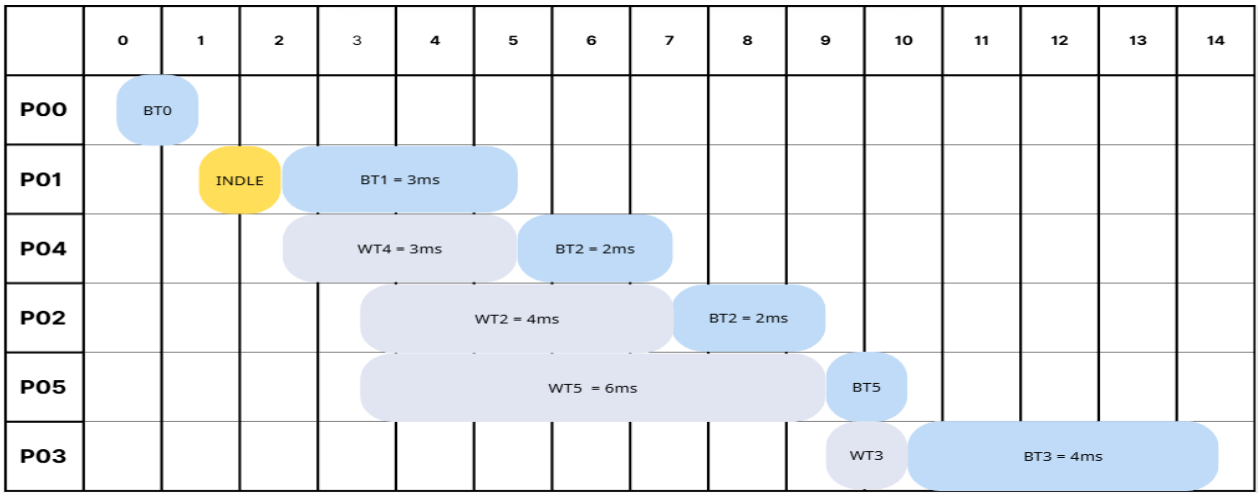


Figure 3.2.2 – Gantt Chart of FCFS Scheduling (Short Burst Time – Uniform Workload)

Based on the results table and details in the Gantt chart, we can see that the CPU is constantly working and there is only a negligible idle time between the two processes P00-P01. The average waiting time is also very small, only about 2.33ms, which shows that the CPU is used thoroughly and efficiently.

The processes arrive almost at the same time and have short bust times, so the queuing delay is almost negligible. From there, System throughput is always maintained stable.

The results show that in an environment with short processes and uniform burst time, the FCFS algorithm helps the CPU achieve high and stable processing performance [3], [6].

3.3. Case 2 – Long Burst Time (Unbalanced Workload)

3.3.1. Description

In this dataset, the workload is intentionally designed to be unfavorable to the FCFS algorithm. This reveals some of its limitations. The main disadvantage is process P04. It has a much longer burst time than the other processes. This situation leads to a phenomenon called convoy effect, where short processes have to wait for the long process to complete.

3.3.2. Results and Observations

Table 3.3.2 – FCFS Scheduling Results (Long Burst Time – Unbalanced Workload)

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT)	Waiting Time (WT)
P00	0	1	1	1	0
P04	2	20	22	20	0
P01	3	3	25	22	19
P02	5	2	27	22	20
P05	5	1	28	23	22
P03	9	1	29	20	19

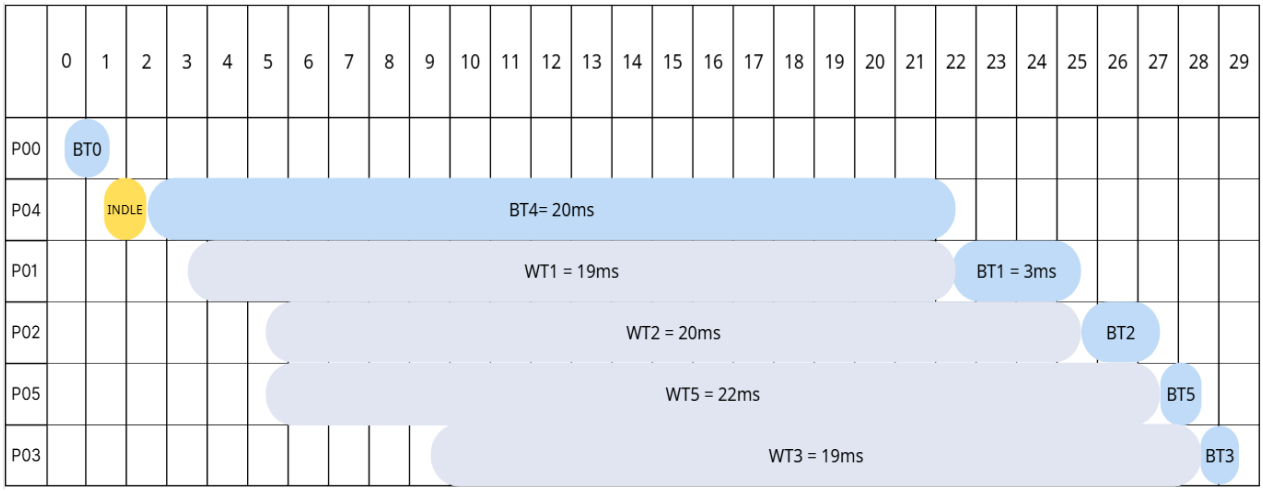


Figure 3.3.2 – Gantt Chart of FCFS Scheduling (Long Burst Time – Unbalanced Workload)

In this case, the waiting time increases sharply by 11ms compared to the previous case. This performance drop comes from process P04 with burst time = 20ms. Since this process is long, the processes that come after it have to wait for a long time.

This case clearly shows the phenomenon of convoy effect [1], [4]. A process that is processed for a long time delays the short processes that come later. From there, the Waiting time increases and the system response decreases.

The results show that in an unbalanced load environment, although the FCFS algorithm ensures fairness in processing processes in order. But FCFS does not optimize throughput, because there is no priority for short processes [4], [6].

3.4. Summary of Findings

From the two simulation cases above, we can draw some main conclusions as follows:

- The simulation program operates exactly according to the principle of the algorithm, non-preemptive and calculates the time indicators correctly.
- For the Uniform Workload environment, FCFS has stable and optimal results, low average waiting time, and maximum utilization of CPU performance.
- In contrast to the Unbalanced Workload environment, FCFS clearly shows inefficiency due to the convoy effect, when long processes delay the short processes that come later.
- These two results show the typical trade-offs of FCFS. Although simple and ensures fairness through sequential processing, FCFS is not suitable for systems with high requirements on response speed or that need to prioritize certain processes.

CONCLUSION

In this chapter, the performance of the **First-Come, First-Served (FCFS)** scheduling algorithm was analyzed and compared with other common CPU scheduling

algorithms such as **Round Robin (RR)**, **Shortest Job First (SJF)**, and **Shortest Remaining Time (SRT)**.

The analysis revealed that FCFS is simple, predictable, and easy to implement, making it suitable for batch processing systems [1], [2]. It ensures fairness by serving processes in the order of their arrival without preemption. However, its main drawbacks include long waiting times for later processes, the convoy effect, and low CPU utilization efficiency when burst times vary significantly.

When compared with other algorithms:

- **Round Robin** improves responsiveness and fairness by assigning fixed time quanta to each process, making it more appropriate for interactive systems, though at the cost of more frequent context switching and higher complexity.
- **Shortest Job First (SJF)** minimizes average waiting and turnaround times by prioritizing shorter jobs, achieving theoretical optimality but potentially starving longer processes and requiring prior knowledge of burst times.
- **Shortest Remaining Time (SRT)** further enhances CPU utilization and responsiveness through preemption, making it highly efficient for dynamic environments, but it introduces complexity and unpredictability due to frequent process switching.

Overall, while FCFS provides a foundation for understanding CPU scheduling, it is generally less efficient in modern, time-sharing systems. More advanced algorithms like SJF, RR, and SRT offer better performance in terms of response time, fairness, and CPU utilization, [3], [4], [5], though they demand more sophisticated control and estimation mechanisms.

REFERENCES

[1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[3] H. K. Omar and Z. Jumamil, "Comparative analysis of the essential CPU scheduling algorithms," presented at the Bulletin of Electrical Engineering and Informatics, 2021, pp. 2940–2948. doi: 10.11591/eei.v10i5.3088.

[4] A. P. U. Siahaan, "Comparison Analysis of CPU Scheduling: FCFS, SJF and Round Robin," presented at the International Journal of Engineering Development and Research, 2016, pp. 124–128.

[5] A. Mohammadjafari and P. Khajouie, "Optimizing Task Scheduling in Heterogeneous Computing Environments: A Comparative Analysis of CPU, GPU, and ASIC Platforms Using E2C Simulator," presented at the IEEE Conference, 2024, pp. 1–12. doi: 10.48550/arXiv.2405.08187.

[6] Q. Li, "Advances In CPU Scheduling Algorithms in Operating Systems," *Highlights in Science, Engineering and Technology*, vol. 120, pp. 8–13, 2024, doi: 10.54097/9g6zbb91.